

Customer Intelligence with AI, ML, and NLP: The Explainability Challenge

Module Module 5 AI-900 - Applied AI, ML, and NLP	Certification Alignment AI-900: Azure AI Fundamentals	Estimated Time Approx. 8 to 12 hours
--	---	--

How to Use This Brief

This project ties together the core topics from the AI-900 module: machine learning classification, natural language processing, Azure Cognitive Services, and Responsible AI principles. You will build a customer escalation prediction model, analyse customer feedback text using NLP techniques, audit your model for fairness, and respond to a mid-project regulatory challenge that is realistic of the constraints analysts face in financial services.

This project runs in Python using Jupyter Notebook. Section D offers both an Azure Cognitive Services path (for those with Azure access) and a Python-only path using open-source NLP libraries. Both paths produce equivalent outputs.

This Project's Mindset

An AI model is not neutral. It reflects the data it was trained on, the features you chose to include, and the objective you optimised for. Your job is not just to build something that works. It is to understand what it encodes, who it affects, and whether it should be deployed.

In regulated industries, an AI system that cannot be explained to a regulator cannot be used. Explainability is not optional.

Your Scenario

You are a Data Analyst at Meridian Financial Services, a UK fintech operating current accounts, savings products, loans, credit cards, and insurance. Customer complaints have been rising, and 26.8% of customers who contact support end up escalating their complaint to the Financial Ombudsman Service (FOS), at a cost of approximately £800 per case in staff time and potential compensation.

The Head of Customer Experience has asked you to build an AI-driven customer intelligence system that can identify at-risk customers before they escalate, understand the themes in customer feedback, and help the business respond faster to emerging issues.

Head of Customer Experience:

"I need to know which customers are likely to escalate before they do. If we can flag them early, we can intervene and save the relationship. I also want to understand what our customers are saying in reviews and tickets - not just the star rating, but the actual themes."

Head of Compliance:

"Any AI-driven decision that affects a customer must be explainable. If the FCA audits us, I need to be able to show them exactly why a customer was flagged and what that flag led to. The black box approach is not acceptable in financial services."

Your Datasets

Four files. meridian_customers.csv and meridian_outcomes.csv form the core ML dataset. meridian_tickets.csv and meridian_reviews.csv contain text for NLP analysis.

meridian_customers.csv - one row per customer

Column	Type	What It Means
customer_id	Text	Unique customer identifier, e.g. CUS-0001. Join key across all four files.
age_band	Text	Customer age group: 18-25, 26-35, 36-45, 46-55, 56-65, 65 plus.
product_type	Text	Primary product held: Current Account, Savings Account, Personal Loan, Home Insurance, Credit Card, Mortgage.
region	Text	UK region: London, South East, North West, Midlands, Scotland, Wales, North East.
primary_channel	Text	How the customer primarily contacts Meridian: Online, Phone, Branch, App.
tenure_months	Number	How long the customer has held an account with Meridian, in months.
account_value_gbp	Number	Total account value in GBP. Proxy for customer segment.
previous_complaints	Number	Number of complaints raised by this customer before the observation window.

meridian_outcomes.csv - one row per customer, contains the target variables

Column	Type	What It Means
customer_id	Text	Join key. Links to meridian_customers.csv.
escalated	True/False	Whether the customer escalated their complaint to the Financial Ombudsman. This is your ML target variable.
churned	True/False	Whether the customer left Meridian within the observation window.
complaint_count_12m	Number	Number of complaints raised in the past 12 months.
nps_score	Number	Net Promoter Score from -100 to +100. Negative scores indicate detractors.

meridian_tickets.csv - one row per support ticket

Column	Type	What It Means
ticket_id	Text	Unique ticket identifier.
customer_id	Text	Join key. Links to meridian_customers.csv.
ticket_date	Date	Date the ticket was raised. Format: YYYY-MM-DD.

category	Text	Ticket category: Billing, Account Access, Fraud Query, Product Query, Complaint, Service Failure, Refund Request.
resolution_time_hrs	Number	Hours taken to resolve the ticket.
resolved	True/False	Whether the ticket was marked as resolved.
ticket_text	Text	Free-text description of the customer issue. This is the primary NLP input.

meridian_reviews.csv - one row per customer review

Column	Type	What It Means
review_id	Text	Unique review identifier.
customer_id	Text	Join key. Links to meridian_customers.csv.
review_date	Date	Date the review was submitted.
star_rating	Number	Star rating from 1 (very negative) to 5 (very positive).
product_type	Text	Product the review relates to.
review_text	Text	Full customer review text. The primary NLP input alongside ticket_text.

SECTION A * Problem Framing and Responsible AI Planning

What This Is

Before building any model or running any NLP pipeline, you must make and document four decisions. In regulated financial services, these decisions form the basis of an AI governance record. Write your answers in markdown cells at the top of your notebook.

Questions to Answer

1. What is the model's intended use - and what is it explicitly NOT for?

Write two to three sentences stating exactly what the escalation prediction model will be used for - for example, triaging customers for proactive outreach. Then write at least two statements about what it must NOT be used for - for example, automatically denying service or applying different pricing to flagged customers. This distinction matters for FCA compliance.

2. What protected characteristics are in the data - and how will you handle them?

The FCA's Consumer Duty requires that AI systems do not produce discriminatory outcomes for customers with protected characteristics. age_band is a proxy for age. Identify any other columns that could act as proxies for protected characteristics. For each one, write one sentence explaining your handling decision: include it, exclude it, or include it only for bias monitoring.

3. How will 'escalation risk' be communicated to staff - and what action does it trigger?

A risk score is only useful if it triggers a specific, proportionate action. Write a short process map - in plain English - describing what happens when a customer is flagged as high escalation risk. Who receives the alert? What action do they take? What is the timeline? This is the human-in-the-loop requirement for responsible AI in financial services.

4. What does failure look like - and what is the cost?

Write the cost of each error type in this specific context. A false negative means a high-risk customer was not flagged and escalated to the FOS at approximately £800 per case. A false positive means a low-risk customer received unnecessary intervention at approximately £25 in staff time. Use these figures in your model evaluation in Section C.

SECTION B * Data Preparation and Exploratory Analysis

What This Is

Load and merge the structured data from `meridian_customers.csv` and `meridian_outcomes.csv`. Conduct EDA to understand the escalation signal before building any model. Do not touch the text data yet - that is Section D.

Tasks

1. Load and merge the structured datasets.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats

customers = pd.read_csv('meridian_customers.csv')
outcomes = pd.read_csv('meridian_outcomes.csv')

# Merge on customer_id
df = customers.merge(outcomes, on='customer_id', how='inner')

print(f'Shape after merge: {df.shape}')
print(f'Escalation rate: {df["escalated"].mean():.1%}')
print(df.info())
```

2. Analyse the escalation signal across key features.

```
# Escalation rate by product type
prod_esc = df.groupby('product_type')['escalated'].mean().sort_values(ascending=False)
print('Escalation rate by product:')
print((prod_esc * 100).round(1))

# Escalation rate by age band
age_esc = df.groupby('age_band')['escalated'].mean()
print('\nEscalation rate by age band:')
print((age_esc * 100).round(1))

# Previous complaints vs escalation
print('\nEscalation rate by previous_complaints:')
print((df.groupby('previous_complaints')['escalated'].mean() * 100).round(1))

# Visualise: escalation rate by product
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
prod_esc.plot(kind='barh', ax=axes[0], color='#2E5FA3')
axes[0].set_title('Escalation Rate by Product', fontweight='bold')
```

```

axes[0].set_xlabel('Escalation Rate')

# Previous complaints boxplot
sns.boxplot(data=df, x='escalated', y='previous_complaints',
            ax=axes[1], palette=['#2E5FA3', '#E65100'])
axes[1].set_title('Previous Complaints by Escalation Status', fontweight='bold')
plt.tight_layout()
plt.savefig('eda_escalation.png', dpi=150)
plt.show()

```

In markdown cells below each analysis block, write one observation and one insight. Use the churn rate contrast (38.8% for escalated vs 13.4% for non-escalated) as a business framing anchor.

SECTION C * Machine Learning - Escalation Risk Classification

What This Is

Build a supervised classification model to predict whether a customer will escalate their complaint. This is a binary classification problem - the same type covered in Session 21 of the module. The target variable is escalated in meridian_outcomes.csv.

Tasks

1. Prepare features and encode categorical variables.

```

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (classification_report, confusion_matrix,
                             roc_auc_score, roc_curve)

# Define features and target
feature_cols = ['tenure_months', 'account_value_gbp', 'previous_complaints',
               'age_band', 'product_type', 'primary_channel', 'region']
target_col = 'escalated'

# Encode categoricals
df_model = pd.get_dummies(df[feature_cols + [target_col]],
                          columns=['age_band', 'product_type',
                                   'primary_channel', 'region'],
                          drop_first=True)

X = df_model.drop(target_col, axis=1)
y = df_model[target_col].astype(int)

print(f'Features: {X.shape[1]}')
print(f'Target balance: {y.mean():.1%} escalated')

```

2. Train two models and compare them.

Train both a Logistic Regression (interpretable) and a Random Forest (higher performance). In regulated financial services, interpretability often matters as much as accuracy. Your Section A decision on intended use will guide which model you recommend.

```

# Split with stratification to preserve class balance
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y)

# Scale features
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# Model 1: Logistic Regression (interpretable)
lr_model = LogisticRegression(
    class_weight='balanced', max_iter=1000, random_state=42)
lr_model.fit(X_train_scaled, y_train)

# Model 2: Random Forest (higher performance)
rf_model = RandomForestClassifier(
    n_estimators=100, class_weight='balanced', random_state=42)
rf_model.fit(X_train_scaled, y_train)

# Evaluate both
for name, model in [('Logistic Regression', lr_model),
                    ('Random Forest', rf_model)]:
    preds = model.predict(X_test_scaled)
    proba = model.predict_proba(X_test_scaled)[:, 1]
    print(f'\n=== {name} ===')
    print(classification_report(y_test, preds,
                               target_names=['Not Escalated', 'Escalated']))
    print(f'AUC: {roc_auc_score(y_test, proba):.3f}')

```

3. Evaluate the business cost of each model at your chosen threshold.

```

# Use your chosen model's probability scores
chosen_model = rf_model # Change to lr_model if you chose Logistic Regression
y_proba = chosen_model.predict_proba(X_test_scaled)[:, 1]

# Test three thresholds
COST_FN = 800 # Cost of missing a high-risk customer (FOS case)
COST_FP = 25 # Cost of unnecessary intervention

print(f'{ "Threshold":<12} { "TP":<6} { "FP":<6} { "FN":<6} { "TN":<6} '
      f' { "Recall":<8} { "Precision":<12} { "Est Cost GBP"}')
print('-' * 80)

for thresh in [0.30, 0.40, 0.50]:
    y_pred = (y_proba >= thresh).astype(int)
    tn,fp,fn,tp = confusion_matrix(y_test, y_pred).ravel()
    precision = tp / (tp+fp) if (tp+fp)>0 else 0
    recall = tp / (tp+fn) if (tp+fn)>0 else 0
    # Scale to full portfolio (test = 20%)
    total_cost = (fn * COST_FN + fp * COST_FP) * 5
    print(f'{thresh:<12} {tp:<6} {fp:<6} {fn:<6} {tn:<6} '
          f' {recall:<8.3f} {precision:<12.3f} GBP {total_cost:,.0f}')

```

In a markdown cell below the cost analysis: state your recommended threshold and the business justification. Reference the specific cost figures from Section A - not just the F1 score.

SECTION D * NLP - Understanding What Customers Are Actually Saying

What This Is

The structured ML model tells you who is at risk. The NLP pipeline tells you why. This section runs sentiment analysis and key phrase extraction on ticket text and customer reviews. Two paths are provided. Complete either Path A (Azure Cognitive Services) or Path B (Python open-source). Both paths produce the same outputs.

Path A - Azure Cognitive Services (recommended if you have Azure access):

Azure Text Analytics provides sentiment analysis (positive/neutral/negative/mixed with confidence scores), key phrase extraction, and named entity recognition via a REST API or the azure-ai-textanalytics Python package. If you are using this path, you will need a free Azure account and a Text Analytics resource created in the Azure portal.

Path B - Python open-source (no Azure account needed):

Use the VADER sentiment analyser (available in the nltk library) for sentiment scoring and spaCy for key phrase and entity extraction. VADER is specifically designed for short, informal text including customer feedback. spaCy provides comparable entity recognition to Azure's service.

Path A - Azure Cognitive Services

Setup: Create the Azure Text Analytics resource

- * Go to portal.azure.com and sign in with a free account.
- * Create a new resource: search for 'Language' and select Azure AI Language.
- * Choose Free tier (F0) - this gives 5,000 transactions per month at no cost.
- * After deployment, copy the Endpoint URL and Key from Keys and Endpoints.

```
# Install: pip install azure-ai-textanalytics
from azure.ai.textanalytics import TextAnalyticsClient
from azure.core.credentials import AzureKeyCredential

# Replace with your actual endpoint and key
ENDPOINT = 'https://your-resource.cognitiveservices.azure.com/'
KEY       = 'your-key-here'

client = TextAnalyticsClient(
    endpoint=ENDPOINT,
    credential=AzureKeyCredential(KEY)
)

# Load a sample of ticket texts (Azure free tier: 5,000 requests)
tickets = pd.read_csv('meridian_tickets.csv')
sample = tickets.head(100) # Start with 100 records

# Run sentiment analysis
docs = list(sample['ticket_text'])
results = client.analyze_sentiment(documents=docs)

sentiments = []
for doc in results:
```

```

        if not doc.is_error:
            sentiments.append({
                'sentiment': doc.sentiment,
                'positive_score': doc.confidence_scores.positive,
                'neutral_score': doc.confidence_scores.neutral,
                'negative_score': doc.confidence_scores.negative,
            })

sentiment_df = pd.DataFrame(sentiments)
print(sentiment_df['sentiment'].value_counts())

```

Azure: Key Phrase Extraction

```

# Extract key phrases from ticket texts
kp_results = client.extract_key_phrases(documents=docs)

all_phrases = []
for doc in kp_results:
    if not doc.is_error:
        all_phrases.extend(doc.key_phrases)

# Count most common key phrases
from collections import Counter
phrase_counts = Counter(all_phrases)
top_phrases = pd.DataFrame(phrase_counts.most_common(20),
                           columns=['phrase', 'count'])

print(top_phrases)

# Bar chart of top phrases
top_phrases.set_index('phrase')['count'].plot(kind='barh', figsize=(10,6))
plt.title('Top 20 Key Phrases in Support Tickets')
plt.tight_layout()
plt.savefig('key_phrases.png', dpi=150)
plt.show()

```

Path B - Python Open-Source NLP

```

# Install: pip install vaderSentiment spacy
# Also run: python -m spacy download en_core_web_sm

from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
import spacy

# Load sentiment analyser
analyser = SentimentIntensityAnalyzer()

# Load NLP pipeline
nlp = spacy.load('en_core_web_sm')

# Run sentiment on ticket text
tickets = pd.read_csv('meridian_tickets.csv')

def score_sentiment(text):
    scores = analyser.polarity_scores(str(text))
    if scores['compound'] >= 0.05: return 'positive', scores['compound']
    elif scores['compound'] <= -0.05: return 'negative', scores['compound']

```

```

else:
    return 'neutral', scores['compound']

tickets[['sentiment','compound_score']] = tickets['ticket_text'].apply(
    lambda t: pd.Series(score_sentiment(t)))

print('Sentiment distribution:')
print(tickets['sentiment'].value_counts())
print(f'\nAvg compound score: {tickets["compound_score"].mean():.3f}')

```

```

# Key phrase and entity extraction with spaCy
def extract_entities(text):
    doc = nlp(str(text))
    orgs = [ent.text for ent in doc.ents if ent.label_ == 'ORG']
    money = [ent.text for ent in doc.ents if ent.label_ == 'MONEY']
    # Noun chunks as key phrases
    phrases = [chunk.text.lower() for chunk in doc.noun_chunks
               if len(chunk.text.split()) >= 2]
    return orgs, money, phrases

# Apply to a sample (spaCy is CPU intensive - use a sample first)
sample = tickets.head(200).copy()
sample[['orgs','money_mentions','key_phrases']] = sample['ticket_text'].apply(
    lambda t: pd.Series(extract_entities(t)))

# Flatten all phrases and count
all_phrases = [p for phrases in sample['key_phrases'] for p in phrases]
top_phrases = pd.DataFrame(
    Counter(all_phrases).most_common(20),
    columns=['phrase','count']
)
print(top_phrases)

```

Required NLP Outputs (complete regardless of Path A or B)

Output	What to Build	In Markdown Below
Sentiment distribution chart	Bar chart: % positive / neutral / negative across all ticket texts	Write one insight on what the overall sentiment pattern tells you about the customer experience
Sentiment by ticket category	Heatmap or grouped bar: avg sentiment score per category	Which category generates the most negative sentiment? Is that the same as the highest-volume category?
Sentiment vs escalation	Boxplot or violin plot: sentiment score by escalated vs not escalated	Does negative sentiment in tickets predict escalation? Quantify the difference.
Top key phrases	Bar chart: top 20 phrases from ticket texts	Do the top phrases match what the Head of Customer Experience would expect? What surprises you?
Review sentiment over time	Line chart: avg review star rating by month	Is the customer experience improving or worsening over time?

SECTION E * Responsible AI - Bias Audit and Fairness Assessment

What This Is

A model that predicts escalation risk may be more or less accurate for different groups of customers. If the model disproportionately flags customers in certain age bands or regions as high risk - without that being justified by the actual escalation rate - it may produce unfair outcomes. This section asks you to check that.

Bias Audit Tasks

1. Check model accuracy by age band and by product type.

```
# Rebuild test set with original features for filtering
test_idx = X_test.index
df_test = df.loc[test_idx].copy()
df_test['y_pred'] = chosen_model.predict(X_test_scaled)
df_test['y_proba'] = chosen_model.predict_proba(X_test_scaled)[:, 1]
df_test['y_true'] = y_test.values

# Accuracy and recall by age band
print('=== Model Performance by Age Band ===')
for band in df_test['age_band'].unique():
    subset = df_test[df_test['age_band'] == band]
    if len(subset) == 0: continue
    acc = (subset['y_true'] == subset['y_pred']).mean()
    recall = (subset[(subset['y_true']==1)][subset['y_pred']==1]).mean()
    n_pos = (subset['y_true']==1).sum()
    print(f' {band:<10} n={len(subset):<4} ' +
          f'accuracy={acc:.3f} recall={recall:.3f} ' +
          f'true positives={n_pos}')

# Same by product type
print('\n=== Model Performance by Product Type ===')
for prod in df_test['product_type'].unique():
    subset = df_test[df_test['product_type'] == prod]
    if len(subset) == 0: continue
    acc = (subset['y_true'] == subset['y_pred']).mean()
    recall = (subset[(subset['y_true']==1)][subset['y_pred']==1]).mean()
    print(f' {prod:<20} n={len(subset):<4} ' +
          f'accuracy={acc:.3f} recall={recall:.3f}')
```

2. Check the False Positive Rate by age band.

A high False Positive Rate for a particular group means that group is disproportionately flagged as at risk when they are not. In financial services, this could constitute discriminatory treatment under the Equality Act 2010.

```
# False Positive Rate = FP / (FP + TN) for each group
# = fraction of non-escalating customers incorrectly flagged
print('=== False Positive Rate by Age Band ===')
for band in sorted(df_test['age_band'].unique()):
    subset = df_test[df_test['age_band'] == band]
    negatives = subset[subset['y_true'] == 0]
    if len(negatives) == 0: continue
    fpr = (negatives['y_pred'] == 1).mean()
    actual_esc = (subset['y_true'] == 1).mean()
    print(f' {band:<10} FPR={fpr:.3f} actual_esc_rate={actual_esc:.3f}')
```

In a markdown cell below the bias audit, write:

- * Which group has the highest False Positive Rate - and is that proportionate to their actual escalation rate?
- * Which group has the lowest recall - meaning the model misses the most real escalations in that group?
- * Based on these findings, would you recommend deploying this model as-is, or are there conditions you would apply?

SECTION F * THE TWIST - The FCA Explainability Requirement

Stop. Read this message from the Head of Compliance before continuing.

"I have had a call from our FCA relationship manager. Following the new Consumer Duty guidance, any AI or algorithmic tool that influences how we treat customers must have an explainability layer - meaning we must be able to show any individual customer why they were flagged, in plain English. I know this is not what you had planned. But we cannot deploy any model without it. I need an explainability section added to your analysis before this can go to the Head of Customer Experience."

What explainability means in practice

SHAP (SHapley Additive exPlanations) is the industry-standard method for explaining individual ML model predictions. It tells you, for each customer prediction, how much each feature contributed to the risk score. A SHAP value of +0.15 for 'previous_complaints' means that the customer's complaint history pushed their risk score up by 15 percentage points compared to the baseline.

Tasks

1. Install SHAP and generate global feature importance.

```
# Install: pip install shap
import shap

# Create explainer for your chosen model
# TreeExplainer is fastest for Random Forest
# LinearExplainer is best for Logistic Regression

if hasattr(chosen_model, 'estimators_'): # Random Forest
    explainer = shap.TreeExplainer(chosen_model)
    shap_values = explainer.shap_values(X_test_scaled)
    # For binary classification, take class 1 (escalated)
    sv = shap_values[1] if isinstance(shap_values, list) else shap_values
else: # Logistic Regression
    explainer = shap.LinearExplainer(chosen_model,
                                     X_train_scaled)
    shap_values = explainer.shap_values(X_test_scaled)
    sv = shap_values

# Global summary: which features drive escalation most?
plt.figure(figsize=(10, 6))
shap.summary_plot(sv, X_test_scaled,
                  feature_names=X.columns.tolist(),
                  plot_type='bar', show=False)
```

```
plt.title('Global Feature Importance - What Drives Escalation Risk?')
plt.tight_layout()
plt.savefig('shap_global.png', dpi=150)
plt.show()
```

2. Explain three individual customer predictions.

Regulators do not want to see a bar chart. They want to understand why a specific customer was flagged. Choose three customers from your test set: one correctly predicted as high risk, one false positive, and one false negative. Generate a SHAP waterfall chart for each.

```
# Find the three cases to explain
df_test_idx = df_test.copy().reset_index(drop=True)
sv_df = pd.DataFrame(sv, columns=X.columns)

# Case 1: True Positive (correctly flagged as high risk)
tp_idx = df_test_idx[(df_test_idx['y_true']==1) &
                     (df_test_idx['y_pred']==1)].index[0]

# Case 2: False Positive (incorrectly flagged)
fp_idx = df_test_idx[(df_test_idx['y_true']==0) &
                     (df_test_idx['y_pred']==1)].index[0]

# Case 3: False Negative (high risk missed)
fn_idx = df_test_idx[(df_test_idx['y_true']==1) &
                     (df_test_idx['y_pred']==0)].index[0]

# Generate waterfall for each case
for label, idx in [('True Positive', tp_idx),
                  ('False Positive', fp_idx),
                  ('False Negative', fn_idx)]:
    plt.figure(figsize=(10, 5))
    shap.waterfall_plot(
        shap.Explanation(
            values=sv[idx],
            base_values=explainer.expected_value
            if not isinstance(explainer.expected_value, list)
            else explainer.expected_value[1],
            data=X_test_scaled[idx],
            feature_names=X.columns.tolist()
        ),
        show=False
    )
    plt.title(f'SHAP Explanation: {label}')
    plt.tight_layout()
    plt.savefig(f'shap_{label.lower().replace(" ", "_")}.png', dpi=150)
    plt.show()
```

3. Write a plain-English explanation for the regulator.

For each of the three SHAP waterfall charts, write a plain-English explanation (approximately 50 words each) that a non-technical FCA examiner could read and understand. Use this structure:

Regulator explanation template:

'Customer [ID] was [flagged / not flagged] as high escalation risk. The three factors that contributed most to this assessment were: [factor 1 with value], [factor 2 with value], and [factor 3

with value]. The model assigned this customer a risk score of [X]%, compared to the average of [Y]%. This assessment [led / did not lead] to a proactive outreach action.'

SECTION G * Power BI AI Insights Dashboard

What This Is

Export your model outputs and NLP results to CSV and build a Power BI dashboard that the Head of Customer Experience can use to monitor escalation risk and sentiment trends. This section integrates the ML and NLP outputs with Power BI's AI-native features.

Tasks

1. Export your model outputs for Power BI.

```
# Export: customer profiles + ML risk scores + NLP sentiment + outcomes
# Join everything on customer_id

# ML outputs
df_test['risk_score_pct'] = (df_test['y_proba'] * 100).round(1)
df_test['risk_band'] = pd.cut(
    df_test['y_proba'],
    bins=[0, 0.3, 0.6, 1.0],
    labels=['Low Risk', 'Medium Risk', 'High Risk']
)

# Combine with customer profile for Power BI
export_df = df_test[[
    'customer_id', 'age_band', 'product_type', 'region',
    'primary_channel', 'tenure_months', 'account_value_gbp',
    'previous_complaints', 'risk_score_pct', 'risk_band',
    'y_true', 'y_pred', 'escalated', 'churned', 'nps_score'
]].copy()

export_df.to_csv('meridian_ml_output.csv', index=False)

# Also export sentiment results
# (join your sentiment_df to tickets and export)
tickets['sentiment_score'] = tickets['ticket_text'].apply(
    lambda t: analyser.polarity_scores(str(t))['compound'])
tickets.to_csv('meridian_nlp_output.csv', index=False)

print(f'ML output: {len(export_df)} rows')
print(f'NLP output: {len(tickets)} rows')
```

2. Build four Power BI report pages.

Page	Audience	Required Visuals
1 - Escalation Risk Overview	Head of Customer Experience	KPI: total high-risk customers. KPI: % flagged this month. Bar: risk band by product type. Map: risk concentration by region. Slicer: product type and risk band.

2 - Customer Detail (Drill-Through)	Customer Ops Team	Individual customer risk score, tenure, complaint history, and NLP sentiment from their most recent ticket. Back navigation button.
3 - Sentiment and NLP Themes	Head of CX and Complaints Manager	Line chart: avg sentiment by month. Bar chart: sentiment by ticket category. Word cloud or bar chart of top key phrases. Star rating trend.
4 - Responsible AI Monitoring	Head of Compliance	Bar chart: escalation rate vs model recall by age band. Table: FPR by group. SHAP global importance chart embedded as image. Compliance statement text box.

3. Use Power BI's built-in AI features on at least one page.

Power BI has native AI visuals that align directly with the AI-900 curriculum. Apply at least one of the following to your dashboard:

AI Feature	Where to Find It	Apply It To
Key Influencers visual	Visualisations pane > Key influencers	What factors most influence whether a customer is in the High Risk band?
Decomposition Tree	Visualisations pane > Decomposition tree	Break down High Risk customers by region, then product type, then age band
Q and A natural language visual	Insert > Q and A	Enable Q and A on Page 1 so the CX team can ask questions like 'Which product has the highest escalation rate this month?'
Smart Narrative	Visualisations pane > Smart narrative	Auto-generate a written summary of the escalation trends on Page 1

SECTION H * AI Reflection and Regulatory Documentation

What This Is

This section produces the two documents a regulated financial services firm would need before deploying an AI system: an AI reflection (internal) and a Responsible AI summary (regulatory-facing). Both are required for submission.

Task 1 - AI Reflection (300 words minimum)

Write a section in your notebook labelled 'AI Reflection - Module 5 Project'. Address all four points:

Required Point	What to Write
What the model does and does not claim	Plain English description of what the escalation model predicts, its AUC score, and the threshold you chose. One clear statement about what the model should never be used for.
What the NLP pipeline found	Two to three specific findings from your sentiment analysis and key phrase extraction. Not generic statements - reference actual data from your outputs.
What the bias audit revealed	Which groups the model performs differently for. Whether those differences are proportionate. Your judgement on whether the model is ready to deploy.

Your view on Responsible AI in financial services	Based on completing this project - not on what you have read. What is the hardest part of responsible AI in practice? What would you do differently with more time?
---	---

Task 2 - Responsible AI Summary (one page)

Write a one-page Responsible AI summary using the following structure. This document is written for the Head of Compliance - not a technical audience.

Section	Content
Model Purpose	One sentence on what the model does and one sentence on what it must not be used for.
Data Used	Plain English description of the four datasets, what features were included, and why protected characteristics were handled as they were.
Performance	Key evaluation metrics in plain English. AUC, recall, and precision translated into business language - not statistical terms.
Bias Findings	Results from the bias audit. Any groups where the model performs materially differently. Your judgement on whether this is acceptable.
Explainability	How SHAP was used. One example in plain English of how an individual prediction can be explained to a customer.
Recommended Conditions for Deployment	Two to three specific conditions that must be met before deployment - for example: human review required for all High Risk flags, quarterly bias monitoring report, customer opt-out process.

SECTION I * Deliverables and Submission

Item	Format	What It Must Contain
1 - Jupyter Notebook	.ipynb	All sections A to H. Every code cell has a markdown cell above with reasoning and below with interpretation. SHAP charts embedded. NLP outputs visualised.
2 - Power BI Dashboard	.pbix	All four report pages. At least one AI-native Power BI visual used. RLS configured for CX Team and Compliance roles.
3 - ML and NLP CSV exports	Two CSV files	meridian_ml_output.csv with risk scores and bands. meridian_nlp_output.csv with sentiment scores per ticket.
4 - Written Submission	.docx or .pdf	Section A framing answers. AI Reflection (300 words). Responsible AI Summary (one page). Three plain-English SHAP explanations.

Section	Must Be Present
A - Framing	Intended use and prohibited uses stated. Protected characteristics identified with handling decisions. Human-in-the-loop process documented. Error type costs stated.
B - EDA	Structured data merged and checked. Escalation rate analysed by product, age band, and complaint history. At least two charts with observation and insight below each.

C - ML Model	Two models trained and compared. Business cost analysis at three thresholds. Recommended threshold with financial justification.
D - NLP	Path A or Path B completed. All five required NLP outputs produced. Sentiment vs escalation comparison included.
E - Bias Audit	Accuracy and recall by age band and product type. FPR by age band. Written judgement on whether the model is ready to deploy.
F - SHAP Explainability	SHAP global importance chart. Waterfall charts for one TP, one FP, and one FN. Three plain-English regulator explanations.
G - Power BI	Four report pages built. ML risk scores and NLP sentiment imported. At least one AI-native Power BI visual used. RLS configured.
H - Documentation	AI Reflection 300 words covering all four required points. Responsible AI Summary one page covering all six sections.

Portfolio Entry

"Built an AI-powered customer intelligence system for a simulated UK fintech: binary classification model (Random Forest and Logistic Regression) to predict complaint escalation, NLP sentiment analysis and key phrase extraction using Azure Cognitive Services and Python, SHAP explainability layer added in response to FCA Consumer Duty compliance requirement, fairness audit across protected characteristic proxies, and Power BI AI dashboard integrating ML outputs, NLP trends, and responsible AI monitoring."